Design and Implementation of a Tournament Branch Predictor Using Verilog HDL

Submitted To

Dr. Dhaval G. Shah
Department of Electronics and Communication Engineering
Nirma University

Submitted By

Khushi Karangiya
Enrollment No: 23BEC082

Kashyap Joshi
Enrollment No: 23BEC084

Course

Computer Architecture

Institution

Institute of Technology

Abstract

Modern high-performance processors depend on instruction-level parallelism (ILP) to execute multiple instructions simultaneously and improve overall performance. However, conditional branch instructions can interrupt the smooth flow of instructions in a pipeline because the processor must determine which path the program will take. This uncertainty creates control hazards, which can reduce pipeline efficiency and slow down execution. To address this issue, processors use branch prediction techniques that attempt to predict the outcome of a branch before it is actually executed.This report focuses on the design and implementation of a tournament branch predictor, inspired by the branch prediction mechanism used in the Alpha 21264 processor architecture. The proposed predictor combines both local and global prediction techniques and uses a chooser predictor to dynamically decide which prediction method should be used for a particular branch. The design is implemented using Verilog HDL and verified through simulation. The results show that the predictor is able to adapt to different branch behavior patterns and achieves better prediction accuracy compared to simpler prediction methods.

Keywords

Branch Prediction, Tournament Predictor, Global History Register, Local Predictor, Verilog HDL, Alpha 21264, Microprocessor Architecture.

1 Literature Survey / State of the Art

Branch prediction is an important technique used in modern processors to reduce performance loss caused by conditional branch instructions. When a processor encounters a branch instruction, it may not immediately know which path the program will follow. This uncertainty can interrupt the smooth flow of instruction execution in pipelined processors. To overcome this problem, several branch prediction techniques have been proposed in the literature to improve prediction accuracy and reduce pipeline stalls.

One of the earliest studies in this area was conducted by James E. Smith (1981) in the paper "A Study of Branch Prediction Strategies." In this work, different branch prediction techniques were analyzed using instruction trace data from several benchmark programs. Smith compared static prediction methods, such as always predicting branches as taken, with dynamic methods that use previous branch outcomes to make predictions. The study showed that dynamic prediction strategies perform better than simple static approaches. In particular, the 2-bit saturating counter predictor was found to significantly reduce mispredictions compared to a 1-bit predictor, especially in loop-based program structures.

Later, Yeh and Patt (1991) introduced the concept of Two-Level Adaptive Branch Predic-

tion, which improved prediction accuracy by using more detailed branch history information. Their approach uses two main structures: a branch history register and a pattern history table. The branch history register stores the outcomes of recent branch executions, while the pattern table records how similar patterns behaved in the past. By combining these two levels of information, the predictor can adapt to the behavior of the running program. Experimental results showed that this technique achieved significantly higher accuracy compared to earlier branch prediction methods.

Further improvements in branch prediction were explored through hybrid branch predictors, which combine multiple prediction techniques. The study "Hybrid Myths in Branch Prediction" investigated how combining different predictors can improve overall performance. The authors observed that different types of branches may be predicted more accurately by different predictors. Therefore, hybrid predictors use a selection mechanism to choose the most suitable predictor for each branch. The study also discussed the issue of aliasing, where multiple branches share the same table entry, which can negatively affect prediction accuracy. Reducing aliasing was identified as an important factor in improving predictor performance.

Another important contribution is presented in the paper "A Comparative Analysis of Schemes for Correlated Branch Prediction." This work focused on understanding why different branch prediction techniques perform differently. The authors proposed a framework for analyzing branch prediction schemes based on how they divide program execution streams and assign predictors. The study also highlighted the importance of branch correlation, where the outcome of a branch can depend on the outcomes of previous branches. By exploiting such correlations, prediction accuracy can be improved further. The paper also emphasized the effectiveness of dynamic predictors such as the 2-bit saturating counter, which can adapt to changes in program behavior during execution.

Overall, these studies show how branch prediction techniques have evolved over time. Early approaches relied on simple static predictions, while later research introduced dynamic, history-based, and hybrid prediction mechanisms. These developments have played a significant role in improving processor performance by reducing the penalties caused by branch instructions in modern pipelined architectures.

2 Limitations of Existing Technologies

Despite improvements in branch prediction, existing techniques have several limitations.

- Static predictors cannot adapt to changing program behavior.
- One-bit predictors frequently mispredict loop exit conditions.
- Global predictors may fail to capture branch-specific patterns.
- Local predictors cannot capture correlations between different branches.

- Larger prediction tables increase hardware complexity and power consumption.

These limitations motivate the use of hybrid prediction techniques such as tournament predictors.

3 Proposed Methodology

The proposed design implements a tournament branch predictor consisting of three main components:

3.1 Global Predictor

The global predictor makes its prediction by looking at the outcomes of recently executed branches across the entire program. Instead of focusing on a single branch, it keeps track of the behavior of multiple previous branches using a Global History Register (GHR). This register stores a sequence of bits representing whether the recent branches were taken or not taken.

The information stored in the global history register is then used to access a pattern history table (PHT) that contains prediction counters. Based on the current global history pattern, the predictor estimates whether the next branch will be taken or not taken. The main advantage of the global predictor is that it can capture correlations between different branches. For example, the outcome of one branch may depend on the result of another earlier branch. By using global history, the predictor can recognize such patterns and improve prediction accuracy.

However, the global predictor may sometimes struggle when different branches share similar history patterns, which can lead to incorrect predictions.

3.2 Local Predictor

The local predictor focuses on the behavior of individual branches rather than looking at global execution history.Each branch instruction maintains its own local history, which records how that specific branch behaved in the past.

The local predictor uses two structures: a Local History Table (LHT) and a Pattern History Table (PHT). The local history table stores the recent outcomes for each branch, while the pattern history table uses this history to predict the next outcome using saturating counters.

The main advantage of the local predictor is that it works well when a branch follows a repeating pattern. For example, a loop branch may be taken several times and then not taken once when the loop ends. The local predictor can easily learn this pattern and predict it correctly most of the time.However, the local predictor cannot capture relationships between different branches because it only considers the past behavior of a single branch.

3.3 Chooser Predictor

The chooser predictor selects whether the global predictor or the local predictor should be trusted for a particular branch. Since different branches may be better predicted by different techniques, combining both predictors can improve overall accuracy. The chooser typically uses a table of saturating counters that track which predictor has been more accurate in the past. When both predictors make a prediction, the chooser decides which one should be used as the final output. Over time, the chooser adapts based on the prediction results and learns which predictor works best for different branch patterns. This dynamic selection process is what makes the tournament branch predictor more effective than using either a global or local predictor alone.

3.4 Prediction Process

1. The program counter indexes the Local History Table.
2. Local history is used to access the Local Pattern History Table.
3. Global history indexes the Global Pattern History Table.
4. The chooser predictor selects either the local or global prediction.
5. The selected prediction becomes the final branch prediction.

4 Block Diagram

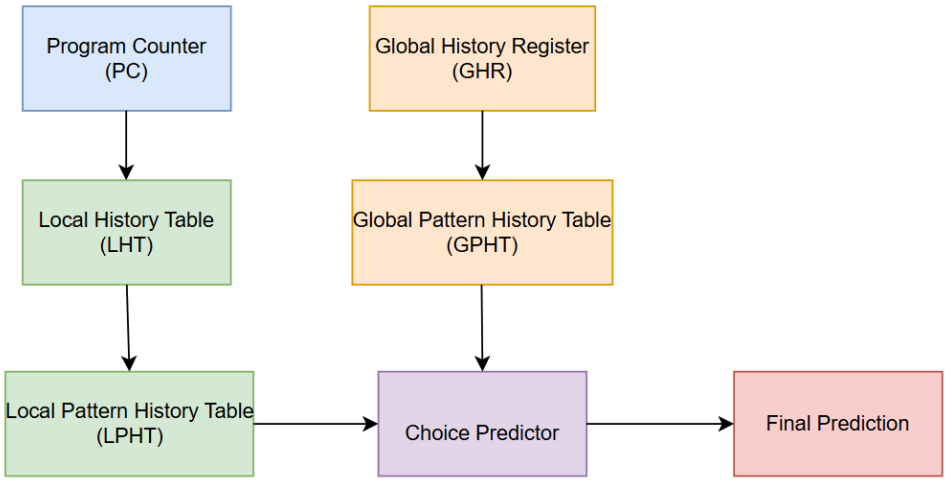

Figure 1: Block diagram of the tournament branch predictor

The architecture consists of a Global History Register, Local History Table, Global Pattern History Table, Local Pattern History Table, and a Choice Predictor that selects the final prediction.

5 State Diagram

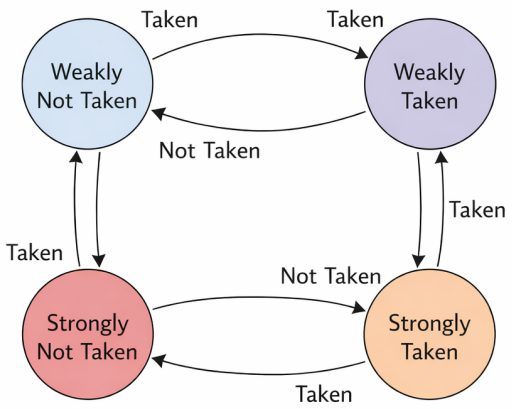

Figure: FSM used in the predictor for a Tournament Branch Predictor

The branch predictor uses a 2-bit saturating counter finite state machine (FSM) consisting of four states: Strongly Not Taken (00), Weakly Not Taken (01), Weakly Taken (10), and Strongly Taken (11). The most significant bit of the counter determines the prediction outcome. If the branch outcome is taken, the counter increments toward the strongly taken state, and if the branch outcome is not taken, the counter decrements toward the strongly not taken state.

6 Simulation and Implementation Results

The predictor was implemented using Verilog HDL and simulated using a ModelSim simulation tool.

Three test scenarios were used to evaluate the design:

6.1 Always Taken Branch Pattern

```
VSIM 2> run -all
# Test 1: Always Taken
# Time=25000 Prediction=0 Actual=1
# Time=35000 Prediction=0 Actual=1
# Time=45000 Prediction=0 Actual=1
# Time=55000 Prediction=0 Actual=1
# Time=65000 Prediction=0 Actual=1
# Time=75000 Prediction=0 Actual=1
# Time=85000 Prediction=1 Actual=1
# Time=95000 Prediction=1 Actual=1
# Time=105000 Prediction=1 Actual=1
# Time=115000 Prediction=1 Actual=1
```

Figure 2: Simulation waveform for always taken branch pattern

The predictor initially mispredicts but eventually learns the correct behavior.

6.2 Always Not Taken Pattern

```
# Test 2: Always Not Taken
# Time=125000 Prediction=1 Actual=0
# Time=135000 Prediction=1 Actual=0
# Time=145000 Prediction=0 Actual=0
# Time=155000 Prediction=0 Actual=0
# Time=165000 Prediction=0 Actual=0
# Time=175000 Prediction=0 Actual=0
# Time=185000 Prediction=0 Actual=0
# Time=195000 Prediction=0 Actual=0
# Time=205000 Prediction=0 Actual=0
# Time=215000 Prediction=0 Actual=0
```

Figure 3: Simulation waveform for always not taken branch pattern

The predictor quickly adapts and predicts the branch correctly after a few cycles.

6.3 Alternating Pattern

```
# Test 3: Altering Pattern
# Time=225000 Prediction=0 Actual=0
# Time=235000 Prediction=0 Actual=1
# Time=245000 Prediction=0 Actual=0
# Time=255000 Prediction=1 Actual=1
# Time=265000 Prediction=0 Actual=0
# Time=275000 Prediction=0 Actual=1
# Time=285000 Prediction=0 Actual=0
# Time=295000 Prediction=0 Actual=1
# Time=305000 Prediction=1 Actual=0
# Time=315000 Prediction=0 Actual=1
# Time=325000 Prediction=1 Actual=0
# Time=335000 Prediction=0 Actual=1
# Time=345000 Prediction=1 Actual=0
# Time=355000 Prediction=0 Actual=1
# Time=365000 Prediction=1 Actual=0
# Time=375000 Prediction=0 Actual=1
```

Figure 4: Simulation waveform for altering branch pattern

The alternating pattern tests the ability of the predictor to learn complex branch behavior. The tournament predictor adapts by utilizing both global and local prediction strategies.

7 Analysis of Results

The simulation results demonstrate that the tournament predictor effectively adapts to different branch behaviors.

Key observations include:

- Initial mispredictions occur because prediction tables start in weak states.
- Over time, the two-bit counters converge to strongly taken or strongly not taken states.
- The chooser predictor dynamically selects the most accurate predictor.
- Hybrid prediction provides better adaptability compared to simple predictors.

8 Conclusion

Branch prediction plays an important role in maintaining high pipeline performance in modern processors. In this report, we presented the design and implementation of a tournament branch predictor inspired by the Alpha 21264 architecture.

The tournament predictor combines both local and global prediction mechanisms, along with a chooser predictor that decides which prediction strategy should be used for a particular branch. This combination allows the system to take advantage of both prediction methods local and global.

The predictor was implemented and tested through simulation with testbench, and the results show that the predictor is able to adapt to different types of branch behavior. During the initial stage, some mispredictions occur while the predictor is learning the branch patterns. However, as execution continues and more history information becomes available, the predictor gradually improves its accuracy and provides more reliable predictions.

References

[1] J. E. Smith, "A Study of Branch Prediction Strategies," in *Proceedings of the 8th Annual International Symposium on Computer Architecture (ISCA)*, 1981, pp. 135–148.

[2] T.-Y. Yeh and Y. N. Patt, "Two-Level Adaptive Training Branch Prediction," in *Proceedings of the 24th Annual International Symposium on Microarchitecture (MICRO-24)*, 1991, pp. 51–61.

[3] A. N. Eden, J. Ringenberg, S. Sparrow, and T. Mudge, "Hybrid Myths in Branch Prediction," in *Proceedings of the International Conference on Information Systems Analysis and Synthesis*, 2002.

[4] C. Young, N. Gloy, and M. D. Smith, "A Comparative Analysis of Schemes for Correlated Branch Prediction," in *Proceedings of the 22nd Annual International Symposium on Computer Architecture (ISCA)*, 1995, pp. 276–286.

[5] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed., Morgan Kaufmann, 2019.

[6] "Branch Prediction Explained," YouTube video. Available: `https://youtu.be/ivLZ8VRucNQ`.

[7] "Branch Prediction and Pipeline Hazards," YouTube video. Available: `https://youtu.be/j4VdblGtnOI`.

A Appendix: Verilog Code

```
module BranchPredictor#(
        parameter PC_bits=6,
        parameter Hist_bits=4
)(
        input wire clk,
        input wire rst,
        input wire [PC_bits-1:0] pc,
        input wire actual_outcome,
        output wire prediction
);
        reg [Hist_bits-1:0] GHR; //Global History Register
        reg [1:0] global_PHT [0:(1<<Hist_bits)-1]; // Global
           Prediction History Table
        reg [Hist_bits-1:0] local_LHT [0:(1<<PC_bits)-1]; // Local
           History Table
        reg [1:0] local_PHT [0:(1<<Hist_bits)-1];       // Local
           Prediction History Table
        reg [1:0] choice_PHT [0:(1<<Hist_bits)-1]; //Choice
           Predictor Table

        wire [Hist_bits-1:0] global_index;
        wire [Hist_bits-1:0] local_hist;

        assign global_index = GHR;
        assign local_hist   = local_LHT[pc];

        wire global_pred;
   wire local_pred;
   wire choose_global;

        assign global_pred  = global_PHT[global_index][1];
   assign local_pred   = local_PHT[local_hist][1];
   assign choose_global = choice_PHT[global_index][1];

        assign prediction = choose_global ? global_pred : local_pred
           ;
        task automatic update_counter;
                inout [1:0] counter;
                input taken;
```

```
                begin
                        if(taken && counter != 2'b11)
                                counter = counter + 1'b1;
                        else if (!taken && counter != 2'b00)
                                counter = counter - 1'b1;
                end
        endtask
        integer i;

        always @(posedge clk or posedge rst) begin
                if(rst)begin
                        GHR<=0;
                        for (i=0;i<(1<<Hist_bits);i=i+1) begin
                                global_PHT[i]<=2'b01;
                                local_PHT[i]<=2'b01;
                                choice_PHT[i]<=2'b01;
                        end
                        for (i=0;i<(1<<PC_bits);i=i+1)
                                local_LHT[i]<=0;

                        end else begin
                        update_counter(global_PHT[global_index],
                           actual_outcome);
                        update_counter(local_PHT[local_hist],
                           actual_outcome);
                        if (global_pred != local_pred) begin
                                if(global_pred == actual_outcome)
                                        update_counter(choice_PHT[
                                           global_index],1'b1);
                                else
                                        update_counter(choice_PHT[
                                           global_index],1'b0);
                        end

                        local_LHT[pc]<={local_hist[Hist_bits-2:0],
                           actual_outcome};
                        GHR<={GHR[Hist_bits-2:0],actual_outcome};
                end
        end
endmodule
```

```
endmodule
```

B Appendix: TestBench Code

```
`timescale 1ns/1ps
module tb_BranchPredictor;
  reg clk;
  reg rst;
  reg [5:0] pc;
  reg actual_outcome;
  wire prediction;

BranchPredictor DUT(
  .clk(clk),
  .rst(rst),
  .pc(pc),
  .actual_outcome(actual_outcome),
  .prediction(prediction)
  );
always #5 clk=~clk;
integer i;
initial begin
  clk=0;
  rst=1;
  pc=6'b000001;
  actual_outcome=0;

  #20;
  rst=0;
  // Test 1: Always Taken
    $display("Test 1: Always Taken");
    for (i = 0; i < 10; i = i + 1) begin
      @(posedge clk);
      actual_outcome = 1;
      $display("Time=%0t Prediction=%b Actual=%b",$time, prediction,
          actual_outcome);
```

```
    end
    // Test 2: Always Not Taken
    $display("Test 2: Always Not Taken");
    for (i = 0; i < 10; i = i + 1) begin
      @(posedge clk);
      actual_outcome = 0;
      $display("Time=%0t Prediction=%b Actual=%b",$time, prediction,
          actual_outcome);
    end
    //Test 3: Alternating Pattern
    $display("Test 3: Altering Pattern");
    for(i=0;i<16;i=i+1) begin
      @(posedge clk);
      actual_outcome=i%2;
      $display("Time=%0t Prediction=%b Actual=%b",$time, prediction,
          actual_outcome);
    end

    #50;
    $finish;
  end
endmodule
```